blocky-stories uploader: upload code

These are the complete files from https://github.com/evanlin23/blocky-stories that do the upload, unedited. GitHub Actions runs `upload.yml` once a day. It runs `scripts/upload.py`, which refreshes my OAuth token from the GitHub secrets, calls `channels.list` (mine=true) and stops unless the ID is my channel's (UCPxmYrlV_DarXarGXWBKB7g), then uploads the video with `videos.insert` as private. `scripts/stories.py` picks the next video.

scripts/upload.py

```
"""Daily upload: the next ready story goes to PandaNanoAdventures as a PRIVATE video, and an issue tells
the owner.

    python scripts/upload.py [--story NAME] [--dry-run]

Ready means: not in uploads.json, the gate is open, and its render release holds the film with a passing
flash
report. The oldest episode number goes first. One story per run (the workflow runs once a day).

With the YouTube secrets set (YT_CLIENT_ID, YT_CLIENT_SECRET, YT_REFRESH_TOKEN; docs/youtube-setup.md)
and the
repo variable YT_AUDITED=true (after the API audit is approved), the film is uploaded with the package's
title and description, made for kids: No. Without them, the issue carries the
download link and the text to paste, for an upload by hand. Either way the owner publishes it.
"""

from __future__ import annotations

import argparse
import datetime as dt
import json
import os
import subprocess
import sys
import tempfile
from pathlib import Path

from stories import ROOT, UPLOADS, all_stories, gate_open, release_assets, release_tag, slug, uploads

from blocky.gate import load_package

REPO = os.environ.get("GITHUB_REPOSITORY", "evanlin23/blocky-stories")


def next_story(done: dict) -> str | None:
    for s in all_stories():
        if s in done:
            continue
        assets = release_assets(s)
        if assets and f"{slug(s)}.mp4" in assets and gate_open(s):
            return s
    return None


def fetch(story: str, d: Path) -> tuple[Path, dict]:
    s = slug(story)
    subprocess.run(["gh", "release", "download", release_tag(story), "--dir", str(d), "--pattern", f"
{s}.mp4",
                    "--pattern", f"{s}.flash.json", "--pattern", f"{s}.contact.png"], check=True)
    return d / f"{s}.mp4", json.loads((d / f"{s}.flash.json").read_text())
```

```
def description(pkg: dict, flash: dict) -> str:
    """The package's description, opening with the measured flashing line when the film has one."""
    text = pkg["description"].strip()
    line = flash.get("description_line")
    if line and not text.startswith("Flashing"):
        text = f"{line}\n\n{text}"
    return text + "\n"


def youtube():
    from google.oauth2.credentials import Credentials
    from googleapiclient.discovery import build

    creds = Credentials(None, refresh_token=os.environ["YT_REFRESH_TOKEN"],
client_id=os.environ["YT_CLIENT_ID"],
                        client_secret=os.environ["YT_CLIENT_SECRET"],
token_uri="https://oauth2.googleapis.com/token",
                        scopes=["https://www.googleapis.com/auth/youtube.upload",
                                "https://www.googleapis.com/auth/youtube.readonly"])
    return build("youtube", "v3", credentials=creds, cache_discovery=False)


def upload(yt, mp4: Path, title: str, desc: str) -> str:
    from googleapiclient.http import MediaFileUpload

    want = os.environ.get("YT_CHANNEL_ID", "").strip()
    if want:   # never upload to the wrong channel
        me = yt.channels().list(part="snippet", mine=True).execute().get("items", [])
        ids = [c["id"] for c in me]
        if want not in ids:
            sys.exit(f"the token belongs to {ids}, not YT_CHANNEL_ID {want}; refusing to upload")
    body = {
        "snippet": {"title": title, "description": desc, "categoryId": "24"},   # Entertainment
        "status": {"privacyStatus": "private", "selfDeclaredMadeForKids": False,
"containsSyntheticMedia": False},
    }
    req = yt.videos().insert(part="snippet,status", body=body,
                             media_body=MediaFileUpload(str(mp4), mimetype="video/mp4", chunksize=16 <<
20,
                                                        resumable=True))
    resp = None
    while resp is None:
        status, resp = req.next_chunk()
        if status:
            print(f"uploaded {int(status.progress() * 100)}%", flush=True)
    return resp["id"]


def issue(title: str, body: str) -> None:
    subprocess.run(["gh", "issue", "create", "--repo", REPO, "--title", title, "--body", body],
check=True)


def record(story: str, entry: dict) -> None:
    done = uploads()
    done[story] = entry
    UPLOADS.write_text(json.dumps(done, indent=1) + "\n")
    subprocess.run(["git", "-C", str(ROOT), "add", "uploads.json"], check=True)
    subprocess.run(["git", "-C", str(ROOT), "commit", "-m", f"uploads: {story}"], check=True)
    subprocess.run(["git", "-C", str(ROOT), "push"], check=True)


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--story", default="")
    ap.add_argument("--dry-run", action="store_true")
    a = ap.parse_args()
    done = uploads()
    story = a.story or next_story(done)
```

```
    if not story:
        print("nothing ready to upload today")
        return
    pkg = load_package(story)
    s = slug(story)
    rel = f"https://github.com/{REPO}/releases/tag/{release_tag(story)}"
    with tempfile.TemporaryDirectory() as td:
        mp4, flash = fetch(story, Path(td))
        if not flash.get("house_bar", {}).get("pass"):
            sys.exit(f"{story}: the film fails the flash bar; not uploading")
        desc = description(pkg, flash)
        title = pkg["title"]
        print(f"{story}: {title!r} ({mp4.stat().st_size / 1e6:.0f} MB)")
        if a.dry_run:
            print(desc)
            return
        today = dt.date.today().isoformat()
        pin = pkg.get("pin", "")
        # API uploads only once the audit is approved (repo variable YT_AUDITED=true): before that,
YouTube locks
        # every API upload as private for good, so the owner uploads by hand from the issue instead
        audited = os.environ.get("YT_AUDITED", "").strip().lower() == "true"
        if audited and all(os.environ.get(k) for k in ("YT_CLIENT_ID", "YT_CLIENT_SECRET",
"YT_REFRESH_TOKEN")):
            vid = upload(youtube(), mp4, title, desc)
            record(story, {"video_id": vid, "uploaded": today, "title": title})
            issue(f"Ready to publish: {title}",
                  f"**{s}** is uploaded to PandaNanoAdventures as **private**.\n\n"
                  f"1. Watch it: https://studio.youtube.com/video/{vid}/edit\n"
                  f"2. Set visibility to Public (or schedule it).\n"
                  f"3. Pin this comment:\n\n> {pin}\n\n"
                  f"If Studio shows it as *locked* private, the API audit isn't approved yet: download
the film "
                  f"from {rel} and upload it by hand with the text below.\n\n"
                  f"<details><summary>Title and
description</summary>\n\n```\n{title}\n\n{desc}```\n</details>\n\n"
                  f"Close this issue once it's public.")
        else:
            record(story, {"manual": True, "issued": today, "title": title})
            issue(f"Upload by hand: {title}",
                  f"**{s}** is ready for PandaNanoAdventures. Automatic uploads are off until the
YouTube API audit is "
                  f"approved (then set the repo variable `YT_AUDITED` to `true`), so upload it by
hand:\n\n"
                  f"1. Download `{s}.mp4` from {rel}\n"
                  f"2. Upload in Studio with the title and description below. Made for kids: **No**. "
                  f"Altered or synthetic content: **No**.\n"
                  f"3. Pin this comment:\n\n> {pin}\n\n"
                  f"```\n{title}\n\n{desc}```\n\nClose this issue once it's public.")


if __name__ == "__main__":
    main()
```

scripts/stories.py

```
"""Shared helpers for the workflows: which stories exist, in what order, and what GitHub already holds
for them."""

from __future__ import annotations

import json
import re
import subprocess
import sys
from pathlib import Path
```

```
ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))
UPLOADS = ROOT / "uploads.json"


def all_stories() -> list[str]:
    """Story folders in upload order: by the episode number in the slug (ep14-... before ep15-...)."""
    out = []
    for d in (ROOT / "stories").iterdir():
        sj = d / "story.json"
        if d.is_dir() and sj.exists():
            slug = json.loads(sj.read_text()).get("slug", d.name)
            m = re.match(r"ep(\d+)", slug)
            out.append((int(m.group(1)) if m else 10 ** 6, d.name))
    return [name for _, name in sorted(out)]


def slug(story: str) -> str:
    return json.loads((ROOT / "stories" / story / "story.json").read_text()).get("slug", story)


def gate_open(story: str) -> bool:
    """`python -m blocky gate` (it rebuilds the timeline first, so the caption checks run too)."""
    r = subprocess.run([sys.executable, "-m", "blocky", "gate", story], cwd=ROOT, capture_output=True,
text=True)
    return r.returncode == 0


def release_tag(story: str) -> str:
    return f"render-{story}"


def release_assets(story: str) -> list[str] | None:
    """Asset names on the story's render release, or None when there's no release."""
    r = subprocess.run(["gh", "release", "view", release_tag(story), "--json", "assets,isPrerelease"],
                       capture_output=True, text=True)
    if r.returncode != 0:
        return None
    d = json.loads(r.stdout)
    if d.get("isPrerelease"):   # a failed render (flash bar) is kept as a prerelease for the record
        return []
    return [a["name"] for a in d["assets"]]


def uploads() -> dict:
    return json.loads(UPLOADS.read_text()) if UPLOADS.exists() else {}
```

.github/workflows/upload.yml

```
# Once a day: the next ready story goes up to PandaNanoAdventures as a private video, and an issue tells
the
# owner it's waiting to be published. Run it by hand from the Actions tab to upload one now.
name: upload
on:
  schedule:
    - cron: "23 15 * * *"   # 15:23 UTC daily
  workflow_dispatch:
    inputs:
      story: {description: "story folder (blank = the next ready one)", default: ""}
concurrency: {group: upload, cancel-in-progress: false}
permissions: {contents: write, issues: write}
jobs:
  upload:
    runs-on: ubuntu-latest
    timeout-minutes: 60
```

```
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.11", cache: pip, cache-dependency-path: requirements*.txt}
      - run: pip install -r requirements.txt -r requirements-upload.txt
      - name: upload
        env:
          GH_TOKEN: ${{ github.token }}
          YT_CLIENT_ID: ${{ secrets.YT_CLIENT_ID }}
          YT_CLIENT_SECRET: ${{ secrets.YT_CLIENT_SECRET }}
          YT_REFRESH_TOKEN: ${{ secrets.YT_REFRESH_TOKEN }}
          YT_CHANNEL_ID: ${{ vars.YT_CHANNEL_ID }}
          YT_AUDITED: ${{ vars.YT_AUDITED }}
        run: |
          git config user.name "github-actions[bot]"
          git config user.email "41898729+github-actions[bot]@users.noreply.github.com"
          python scripts/upload.py --story "${{ inputs.story }}"
```

requirements-upload.txt

```
google-api-python-client
google-auth
```